

Backend Services Reference

This document covers the service and utility layer between the Express handlers (`api.ts`) and the raw database queries (`DatabaseOperationsServices/`).

For handler-to-endpoint mappings, see [API SPECIFICATION.md](#). For database schema, see [DATABASE_SCHEMA.md](#).

File Map

```
backend/src/
├─ api/
│   ├── api.ts                - Express handlers (one per endpoint)
│   ├── interfaces/api.interfaces.ts - Request/response TypeScript types
│   └── Utilities/
│       ├── api.utilities.ts    - Shared handler utilities
│       └── reshape.Utilites.ts - Legacy type conversion (misspelled filename)
├─ middleware/
│   └── middleware.ts           - JWT authentication middleware
├─ services/
│   ├── game.service.ts        - Game session orchestration
│   ├── route.service.ts       - Route lifecycle operations
│   └── statistics.service.ts   - ❌ Deleted – stats handlers call DB query
files directly from api.ts
├─ gameServiceUtils/
│   ├── CheckSession.service.ts - Session validation helper
│   └── PlayeridTypeCheck.ts     - Player ID type guard
├─ db/
│   ├── prisma.ts              - Prisma client singleton + adapter selection
│   └── DatabaseOperationsServices/
│       ├── Read/              - Atomic read query classes
│       ├── Update/            - Atomic update query classes
│       ├── Create/            - Atomic create query classes
│       └── Transaction/        - Multi-step atomic operations
```

game.service.ts

Path: `backend/src/services/game.service.ts`

Orchestrates player session start and route completion. Called directly by the `startNewGame` and `completeRoute` handlers.

`startOrJoin(sessionId, playerId?)`

Ensures a `PlayerSession` exists for the player, then returns their active route (allocating one if needed).

1. `validateSession(sessionId)` – throws if session doesn't exist
2. `validatePlayerId(playerId)` – throws if playerId is not a number
3. `GetOrCreatePlayerSession.getOrCreate(sessionId, playerId)` – idempotent
4. `getActiveRouteForPlayerSession(playerSessionId)` – check for in-progress route

5. If no active route: getNextRouteForPlayerSession(playerSessionId, sessionId) – allocate
6. Return RouteDTO → mapRouteDTOToLegacyRoute()

Returns: Legacy Route shape (see [SHARED_TYPES.md](#))

completeRouteAndAdvance(sessionId, playerId?, completionMs?)

Marks the active route complete, collects stats, and allocates the next route.

1. validateSession + validatePlayerId
2. GetOrCreatePlayerSession
3. completeActiveRouteForPlayerSession(playerSessionId, completionMs)
4. countCompletedRoutes(playerSessionId)
5. getCompletedPollinators(playerSessionId)
6. getNextRouteForPlayerSession(playerSessionId, sessionId) – may return null if all done
7. Return { newActiveRoute, routesCompleted, completedPollinators }

route.service.ts

Path: backend/src/services/route.service.ts

Low-level route lifecycle operations. Called by `game.service.ts` .

completeActiveRouteForPlayerSession(playerSessionId, completionMs?)

Sets `wasCompleted = true` , `completedAt = now()` , and `completionMs` on the player's current `RouteCycle` .

countCompletedRoutes(playerSessionId)

Counts `RouteCycle` rows where `wasCompleted = true` for the player session.

getCompletedPollinators(playerSessionId)

Returns an array of pollinator names from all completed routes. Fetches the last fact node of each completed route to get the pollinator name.

pickRandomRouteId(playerSessionId, sessionId)

Selects the next route for the player using cycle-aware logic:

1. Get current cycle number via `getLatestCycleNumber()`
2. Find all routeIds already assigned in the current cycle (`RouteCycle.cycleNumber = currentCycle`)
3. Find all available route IDs in the system
4. Subtract already-assigned routes from available
5. If unseen routes remain → pick randomly from unseen
6. If all routes seen → increment cycle, pick randomly from ALL routes
7. Return selected routeId

Route selection uses `crypto.randomInt` (cryptographically secure).

buildWhere(playerSessionId, cycleNumber)

Helper that constructs Prisma `where` filters for `RouteCycle` queries.

statistics.service.ts

Status: DELETED

This file previously contained methods for route statistics aggregation that were planned but disabled (wrapped in block comments). It has since been removed from the codebase entirely.

The Route Statistics feature is now implemented directly via atomic DB query classes in `DatabaseOperationsServices/Read/` and exposed through 20 dedicated handlers in `api.ts`. See [ROUTE_STATISTICS_SPEC.md](#) for the full spec and [API_SPECIFICATION.md](#) for endpoint details.

CheckSession.service.ts

Path: `backend/src/services/gameServiceUtils/CheckSession.service.ts`

Single method: `requireSession(sessionId)`.

Calls `HasSessionService.hasSession(sessionId)`. Throws an error if the session doesn't exist or isn't valid for today.

`HasSession` checks two things:

1. The session exists in the `Session` table.
2. `sessionDate` equals today in the `America/New_York` timezone.

Timezone note: The session validity check is hardcoded to `America/New_York`. This affects deployments in other timezones — a session created with today's date in New York may not be valid if the server runs in a different timezone offset.

PlayeridTypeCheck.ts

Path: `backend/src/services/gameServiceUtils/PlayeridTypeCheck.ts`

Single method: `requirePlayerId(playerId?)`.

Throws `{ error: "Missing or invalid playerId" }` if `playerId` is not of type `number`. Used as a type guard before all game operations.

API Utilities (`api.utilities.ts`)

Path: `backend/src/api/Utilities/api.utilities.ts`

`toNumber(value)`

Safely converts any value to a number. Returns `null` if conversion fails or result is NaN.

`getSessionIdFromReq(req)`

Extracts `sessionId` from `req.auth` (injected by JWT middleware).

`getPlayerIdFromReq(req)`

Extracts `playerId` from `req.auth` .

`generateUniquePlayerId(prisma, sessionId)`

Generates a collision-free numeric player ID:

1. Pick random integer in range 1–2,000,000,000
2. Check if `(sessionId, playerId)` already exists in `PlayerSession`
3. If collision, retry — max 300 attempts
4. Throws if 300 attempts exhausted

`generateSessionId()`

Generates a session ID based on current date in Eastern timezone: format `YYMMDDNNN` where `NNN` is a zero-padded counter (starts at 0).

Reshape Utilities (`reshape.Utilites.ts`)

Path: `backend/src/api/Utilities/reshape.Utilites.ts`

(Note: filename has a typo — "*Utilites*" not "*Utilities*")

`mapRouteDT0ToLegacyRoute(route)`

Converts the rich `RouteDT0` type (returned by DB services) into the legacy `Route` shape expected by the frontend.

```
RouteDT0 {
  id: number, name: string, sourcePath: unknown,
  nodes: [{ id, key, text, position }],
  facts: [{ nodeId, text, pollinator }]
}
↓
Route {
  routeId: string,           ← String(route.id)
  routeNodes: [{ id: string, text: string }], ← from nodes
  factNodes:  [{ id: string, text: string, pollinator: string }] ← from facts
}
```

Note: `RouteNode.id` in the legacy shape is the node's `key` (the coordinate string like `"3.2"`), not the numeric DB id.

JWT Middleware (`middleware.ts`)

Path: `backend/src/middleware/middleware.ts`

`authenticateJWT(req, res, next)`

Express middleware applied to all game endpoints that require authentication.

1. Extracts `Authorization: Bearer <token>` from request headers.
2. Verifies the token using `JWT_SECRET` and the `HS512` algorithm.

- 3. On success: injects `req.auth = { sessionId: number, playerId: number }` and calls `next()` .
- 4. On failure: returns `401 Unauthorized` .

JWT payload shape: `{ sessionId: number, playerId: number }`

Prisma DB Module (`prisma.ts`)

Path: `backend/src/services/db/prisma.ts`

Exports a single `prisma` singleton. Detects the database type from `DATABASE_URL` at runtime:

DATABASE_URL prefix	Adapter used
postgresql://	@prisma/adapters-pg
mysql://	@prisma/adapters-mariadb

Both adapters must be installed (`npm install` includes both). The adapter selection uses a simple string prefix check — if `DATABASE_URL` doesn't match either prefix, the module will throw at startup.

Database Operation Services

Atomic query classes in `backend/src/services/db/DatabaseOperationsServices/` :

Read/

File	Class	
GetpollinatorNames.ts	GetPollinatorNames	getf
LoadRoutedT0.service.ts	LoadRoutedT0	load
getActiveRouteForPlayerSession.service.ts	—	getf
getChildrenandAdultsbySession.ts	—	getC

getSessionSandPlayeridsbyChildSize.ts	—	getf
getSessionandPlayeridbyAdultsize.ts	—	getf
getSessionidsandplayeridsbyfamilysize.ts	—	getf
getChildadultdatabystartandend.ts	—	getf
getChildnumandadultdatetoforever.ts	—	getf
getChildnumandadulttoenddate.ts	—	getT
hasSession.service.ts	HasSessionService	hasS
getTotalRoutesCompleted.ts	GetTotalRoutesCompleted	getT
getTotalConnectedUsers.ts	GetTotalConnectedUsers	getT
getUsersStartedMoreThanOneRoute.ts	GetUsersStartedMoreThanOneRoute	getT
getUsersCompletedMoreThanOneRoute.ts	GetUsersCompletedMoreThanOneRoute	getT
getUsersNeverStartedRoute.ts	GetUsersNeverStartedRoute	getT
getuserswhostartedbutdidnotfinishanyroute.ts	—	getu
getTotalReplays.ts	GetTotalReplays	getT

getAvgRoutesPerUserBySession.ts	GetAvgRoutesPerUserBySession	get/
getAvgRoutesPerUserByDateRange.ts	GetAvgRoutesPerUserByDateRange	get/
getAvgRoutesPerSessionBySession.ts	GetAvgRoutesPerSessionBySession	get/
getAvgRoutesPerSessionByDateRange.ts	GetAvgRoutesPerSessionByDateRange	get/
getReplayRatioBySession.ts	GetReplayRatioBySession	getf
getReplayRatioByDateRange.ts	GetReplayRatioByDateRange	getf
getTotalRoutesCompletedByDateRange.ts	GetTotalRoutesCompletedByDateRange	getT
getTotalConnectedUsersByDateRange.ts	GetTotalConnectedUsersByDateRange	getT

getUsersStartedMoreThanOneRouteByDateRange.ts	GetUsersStartedMoreThanOneRouteByDateRange	getl endl
getUsersCompletedMoreThanOneRouteByDateRange.ts	GetUsersCompletedMoreThanOneRouteByDateRange	getl endl
getUsersNeverStartedRouteByDateRange.ts	GetUsersNeverStartedRouteByDateRange	getl endl
getUsersStartedButNotFinishedByDateRange.ts	GetUsersStartedButNotFinishedByDateRange	getl endl
getTotalReplaysByDateRange.ts	GetTotalReplaysByDateRange	getT

Update/

File	Method	Description
addChildrenadults.ts	addChildrenAdults(sessionId, playerId, numChildren, numAdults)	Set numChildren/numAdults on PlayerSession

Create/

File	Method	Description
getOrCreatePlayerSession.service.ts	getOrCreate(sessionId, playerId)	Idempotent PlayerSession creation

Transaction/

File	Method	Description
------	--------	-------------

<code>getNextRouteForPlayerSession.service.ts</code>	<code>getNextRoute(playerSessionId, sessionId)</code>	Allocate next route with retry loop
--	---	-------------------------------------

`getNextRoute` concurrency handling:

- Uses `prisma.$transaction` for the route allocation.
- Picks a random route, creates a `RouteCycle` row.
- If a `P2002` (unique constraint violation) is thrown (concurrent request allocated same route), it retries — max 300 attempts.
- This handles the case where two tabs or devices share a player session and race to get the next route.